

Documento de Visión y Descripción Pormenorizada: Imrryr OS

1. Visión Conceptual del Sistema

El Imrryr OS es un ecosistema de software local-first diseñado para operar como el núcleo cognitivo, operativo y administrativo de una empresa unipersonal o de estructura descentralizada.

A diferencia de los asistentes comerciales tradicionales o las plataformas SaaS en la nube, este sistema se concibe como un **arnés de software privado y agnóstico** que se ejecuta localmente en el hardware del usuario. El sistema actúa como un "director de orquesta" que consume modelos de inteligencia artificial avanzados a través de APIs intercambiables, delegando subtarefas complejas a micro-agentes especializados que operan de forma atómica y en paralelo.

El propósito fundamental del sistema es unificar tres áreas críticas del flujo empresarial que suelen estar fragmentadas:

1. **La Gestión del Conocimiento y Creatividad:** Capturar ideas espontáneas (semillas), refinarlas y contextualizarlas con proyectos existentes.
2. **La Automatización Operativa y Financiera:** Ejecutar tareas rutinarias en segundo plano (como el control de gastos bancarios mediante correos y el rastreo de fondos públicos).
3. **El Soporte Técnico Continuo:** Actuar como un copiloto de desarrollo que entiende la arquitectura completa de los proyectos de software y museografía interactiva, manteniéndose actualizado mediante un esquema de autodocumentación automatizada.

2. Pilares de Diseño y Filosofía de Arquitectura

Para garantizar la viabilidad a largo plazo y la escalabilidad comercial del sistema, el desarrollo se rige bajo los siguientes principios técnicos fundamentales:

- **Soberanía de Datos e Independencia (Local-First):** Toda la memoria dura, las bases de datos vectoriales, el historial de gastos y los archivos de proyectos residen exclusivamente en el disco duro de la máquina local. El sistema no depende de una base de datos centralizada en la nube, lo que garantiza privacidad absoluta y elimina los costos de mantenimiento de servidores de terceros.
- **Abstracción y Neutralidad de Modelos (Agnóstico):** El núcleo del código no está acoplado a ningún proveedor de IA. Utilizando capas de traducción universal, el sistema puede alternar instantáneamente entre diferentes "cerebros" comerciales o modelos de código abierto locales corriendo en la GPU del usuario según la complejidad de la tarea o la sensibilidad de los datos.

- **Arquitectura Descentralizada y Portable (Modelo Franquiciable):** El sistema se diseña como un nodo autónomo. Si un colaborador o cliente requiere utilizar el AI-OS, se le distribuye una copia limpia del arnés. Cada instalación se configura de forma independiente con las credenciales y rutas locales de su respectivo usuario, permitiendo un modelo de distribución comercial de "marca blanca" sin cruce de datos ni interconexión obligatoria.
- **Mitigación del Colapso Cognitivo (Anti-Context Bloat):** Para evitar que el exceso de herramientas y datos diluya la atención del LLM, el arnés implementa un sistema de divulgación progresiva. Los agentes solo "ven" las herramientas y los fragmentos de información estrictamente necesarios para cumplir su tarea inmediata. Al finalizar, descargan esa memoria de trabajo hacia archivos de estado permanentes.

3. El Modelo de Interacción y Experiencia Omnicanal

El usuario interactúa con el ecosistema a través de dos puntos de acceso complementarios que forman un sistema nervioso integrado:

- **El Entorno de Escritorio (Dashboard Modular):** Una interfaz gráfica de usuario local basada en una cuadrícula modular personalizable. El diseño visual se alinea con una estética oscura, minimalista y lofi, proporcionando un entorno de trabajo onírico y de baja fricción cognitiva. Cada módulo visual funciona de forma independiente (un bloque para finanzas, otro para semillas creativas, otro para el estado de repositorios), permitiendo al usuario reconfigurar su panel de control según sus prioridades del día.
- **El Canal Móvil Asincrónico (Extensión de Bolsillo):** La interacción no se limita al computador. Mediante una pasarela de comunicación ligera (como un bot privado), el usuario puede interactuar con el cerebro de la empresa desde su teléfono móvil mediante notas de voz o texto. Este canal permite volcar ideas en ruta, solicitar resúmenes de documentos complejos en formato de audio, ejecutar acciones remotas en el PC y recibir alertas proactivas o recordatorios push estructurados.

4. Estrategia de Memoria y Autodocumentación de Proyectos

Para mantener la eficiencia de costos por token y la velocidad de respuesta, el sistema aborda la gestión de proyectos de manera estructural. Cada iniciativa (ya sea un software o un diseño de exhibición digital) cuenta con un archivo maestro de estado en texto plano.

La IA actualiza este documento al final de cada sesión de trabajo mediante un proceso automatizado, registrando las decisiones lógicas, tecnologías implementadas y variables creadas. De este modo, el sistema elimina la necesidad de procesar historiales de chat interminables; cada nueva interacción arranca con un contexto limpio y ultra-condensado, leyendo simplemente el archivo de arquitectura actualizado.

He terminado la descripción pormenorizada de la visión y la arquitectura base del proyecto. Quedo atento a tu señal para proceder, en nuestra siguiente interacción, con la numeración y el desglose detallado de cada uno de los módulos, sistemas y herramientas que compondrán el AI-OS.

¿Qué hará el sistema?

1. El Enrutador Central (El Cerebro Director)

- **Qué hace:** Es la puerta de entrada de todo el sistema. Recibe tu instrucción (ya sea por voz desde el celular o escrita en el PC), analiza la intención y decide qué Agente debe trabajar y qué herramientas habilitarle.
- **Cómo lo hace:** Utiliza un framework de orquestación para crear un flujo de decisiones.
-

2. El Registro de Skills y MCPs (La Caja de Herramientas)

- **Qué hace:** Almacena todas las acciones físicas que la IA puede hacer en el mundo real (mandar un mail, crear un archivo, leer una web).
- **Dónde están:** Viven en una carpeta local llamada `/skills`. Son scripts atómicos de Python muy sencillos (de 20 a 50 líneas cada uno).
- **Cómo funcionan los MCPs (Model Context Protocol):** Esta es la tecnología estándar actual. En lugar de que la IA intente adivinar cómo usar tu PC, cada *skill* tiene un "manual de instrucciones" en formato JSON. El Enrutador lee este manual y le dice al agente: *"Tienes esta herramienta para leer Gmail, estos son los parámetros que pide"*.
- **Ejemplos de Skills guardados aquí:** `leer_inbox_gmail()`, `crear_evento_calendario()`, `ejecutar_script_python()`, `convertir_texto_a_audio()`.

3. La Bóveda de Conocimiento (Memoria y Autodocumentación)

- **Qué hace:** Evita que el sistema colapse por exceso de memoria (*Context Bloat*) y mantiene separados los tipos de información.
- **Partes del sistema:**
 - **Memoria Vectorial (RAG):** Usa **ChromaDB** (una base de datos local). Aquí van tus PDFs, libros y manuales de museografía o tecnología. El

sistema convierte el texto en coordenadas matemáticas para buscar conceptos al instante.

- **Memoria Relacional (Gastos):** Usa **SQLite** (una base de datos ligera en un solo archivo). Aquí se guardan filas y columnas precisas con tus compras, montos y fechas.
- **Archivos de Estado (Autodocumentación):** Archivos `.md` (Markdown) que se guardan dentro de la carpeta de cada proyecto de software. Al final del día, un script sobrescribe este archivo resumiendo qué tecnologías usaste y qué falta, para que al día siguiente la IA solo lea ese archivo y se ponga al día en 1 segundo.

4. El Escuadrón de Micro-Agentes (Los Especialistas)

Cada agente es un pequeño *prompt* maestro (una personalidad) emparejado con permisos para usar *skills* específicos. No se mezclan entre sí.

4.1 Agente Financiero (Control Bancario)

- **Qué hace:** Automatiza la contabilidad de VirtualBrain.
- **Cómo funciona:** Cada vez que inicias el PC (mediante un *cron job* local), usa el skill `leer_gmail`. Busca correos de tus bancos, extrae el comercio y el monto, y los inyecta en la base de datos SQLite.
- **Capacidad extra:** Si le preguntas "*¿Cómo voy con los gastos de software este mes?*", lee la SQLite y te genera un reporte en texto.

4.2 Agente Investigador (Scraper de Fondos)

- **Qué hace:** Busca dinero y oportunidades para tus proyectos.
- **Cómo funciona:** Tiene un skill de *web scraping* (usando librerías de Python como BeautifulSoup o Playwright). Todos los días a una hora específica, entra a los portales de Sercotec, Corfo, etc., lee las bases de los nuevos fondos, las cruza con tu archivo de perfil (donde dice que haces tecnología y museografía), y si hay *match*, te genera un resumen.

4.3 Agente Codificador (Tu CTO Adjunto)

- **Qué hace:** Te ayuda a programar, crear herramientas y depurar errores.
- **Cómo funciona:** Tiene acceso de lectura a tus repositorios locales de código. Cuando le pides una mejora, redacta el código y usa un entorno seguro (un contenedor Docker ligero) para probar que funcione sin romper tu PC. Luego, te entrega el fragmento listo para que lo pegues o lo revises.

4.4 Agente Creativo y de Semillas (Gestor de Ideas)

- **Qué hace:** Incuba tus ideas.

- **Cómo funciona:** Si le mandas una idea suelta, el agente la recibe, le asigna etiquetas (ej. [hardware], [videojuegos], [red social]), evalúa si sirve para un proyecto actual, y la guarda en la carpeta `/semillas`.

5. La Pasarela Móvil (App de Bolsillo vía Wsp)

- **Qué hace:** Conecta tu teléfono con tu computador en casa, sin necesidad de programar una app móvil desde cero.

Opción 1: La Vía Local-First (Librerías de Automatización Web)

En la comunidad de desarrollo, para proyectos personales o independientes, se usan mucho herramientas como `whatsapp-web.js` (Node.js) o librerías equivalentes en Python.

- **Cómo funciona:** El sistema levanta un servicio en tu computador que simula ser una sesión de WhatsApp Web. La primera vez, el sistema te muestra un código QR en tu dashboard local, lo escaneas con tu teléfono (igual que cuando abres WhatsApp en el navegador) y listo.
- **Ventajas:** Es 100% gratuito, no necesitas registrar una empresa en Meta, no dependes de servidores externos y puedes enviar/recibir audios, imágenes y textos de forma ilimitada y directa con tu propio número actual. Encaja perfecto con la idea de empaquetar el sistema y que cada colaborador escanee su propio QR en su máquina.

Opción 2: La Vía Oficial (WhatsApp Cloud API de Meta)

Es la API oficial que ofrece Meta para desarrolladores y empresas.

- **Cómo funciona:** Creas una cuenta de desarrollador en Meta, vinculas un número de teléfono y configuras un *Webhook* en tu servidor local para que, cada vez que envíes un mensaje a ese número, Meta le avise a tu computador.
- **Ventajas:** Es extremadamente estable y no depende de tener una sesión web abierta. Meta regala 1,000 conversaciones al mes de forma gratuita, lo cual es más que suficiente para tu uso personal y el de tus primeros clientes.

¿Cómo se integra WhatsApp en el listado de módulos?

Reemplazamos por completo el módulo de Telegram por el **Módulo de Extensión Móvil (WhatsApp Gateway)**. Su funcionamiento exacto con tus necesidades sería el siguiente:

1. **Notas de voz a ideas (Semillas):** Vas en la calle, grabas un audio en WhatsApp diciendo una idea para VirtualBrain y lo envías. Tu PC recibe el archivo `.ogg` del audio, lo procesa localmente con un modelo de transcripción (como Whisper) y el Agente Creativo lo guarda etiquetado en tu carpeta de `/semillas`.

2. **Lectura remota (Tu Podcast Privado):** Si el agente encuentra un fondo concursable, te envía un resumen por texto a WhatsApp. Si le respondes "*Léemelo en audio*", tu PC genera un archivo de voz con IA y te lo envía de vuelta como mensaje de voz para que lo escuches mientras caminas o manejas.
3. **Gestión de Alarmas y Recordatorios:** Le escribes por WhatsApp: "*Recuérdame revisar el presupuesto a las 5 PM*". El sistema procesa la orden y, a las 5 PM en punto, tu computador te envía un mensaje proactivo a tu WhatsApp diciendo: "*Es hora de revisar el presupuesto*".

Al diseñar el sistema de forma modular mediante las **Carpetas Dinámicas** que aprobamos, este conector de WhatsApp es simplemente un archivo dentro de la capa `/gateway`. Si el día de mañana un cliente no quiere usar WhatsApp y prefiere otra aplicación, solo se cambia ese archivo por el conector correspondiente, sin tocar la base de datos de gastos, ni el agente codificador, ni tus ideas.

6. El Dashboard Local (La Interfaz de Escritorio)

- **Qué hace:** Tu centro de mando visual cuando estás frente al computador.
- **Cómo funciona:** Es un servidor web local (hecho en FastAPI o Node.js) que muestra una página HTML/JS en tu navegador (ej. yendo a `localhost:3000`).
- **Estética y Modularidad:** Usando *vibe coding*, generarás un diseño en CSS con estética oscura, melancólica, minimalista. El código usará librerías de componentes arrastrables (Drag & Drop) para que puedas mover los "widgets" (el gráfico de gastos, la lista de semillas, el terminal del Agente Codificador) como piezas de un tablero.

7. Agente Comercial y CRM (Cotizador Automático)

- **Por qué faltaba:** Hablamos de gestionar tus proyectos, pero olvidamos cómo entra el dinero.
- **Qué hace:** Lee correos de clientes potenciales, analiza sus requerimientos y cruza esa info con tu base de datos de precios/proyectos pasados para redactar propuestas formales y cotizaciones en PDF.
- **Para quién es útil:** Vital para ti, y **el módulo más vendible** si se lo ofreces a otra persona.

8. Agente de Presencia (Redes y Web)

- **Por qué faltaba:** Lo mencionamos al principio pero no lo listamos.
- **Qué hace:** Cuando terminas un hito, usa un *skill* para conectarse a tu CMS (como WordPress o el código de tu web) y a la API de LinkedIn/X. Redacta el caso de éxito y lo publica automáticamente.

9. El Guardia de Seguridad (Módulo de Observabilidad)

- **Por qué es estándar en la comunidad:** En los foros (como Reddit/LocalLLaMA), todos los que arman sistemas agénticos advierten sobre el consumo descontrolado de APIs o bucles infinitos.
- **Qué hace:** Un micro-agente silencioso que monitorea. Si el Agente Investigador lleva 5 minutos atascado leyendo una web y gastando *tokens*, el Guardia detiene el proceso y te avisa al celular.

10. El Bus de Eventos (Webhooks en tiempo real)

- **Por qué es estándar:** No basta con que el sistema revise cosas "al encender el PC". La comunidad usa arquitecturas orientadas a eventos.
- **Qué hace:** En lugar de preguntar "¿hay correos nuevos?", el sistema expone un *Webhook* (una dirección web invisible). Cuando el banco te manda el recibo de compra, ese correo "golpea" el webhook y el Agente Financiero reacciona en ese mismo milisegundo.

PARTE 2: Modularidad Extrema (El modelo "App Store" interno)

Para lograr lo que quieres —poder venderle esto a un panadero, a un abogado o a un arquitecto quitando y poniendo piezas sin reescribir todo el código—, el sistema no puede ser un bloque sólido. Debe funcionar como un sistema de "**Carpetas Dinámicas**" (**Hot-Plugging**).

Se diseña de la siguiente manera:

A. El Enrutador es Agnóstico (No sabe quiénes son los agentes) El código principal (el Enrutador) no tiene los nombres de los agentes escritos a fuego. En su lugar, cuando el programa se enciende, hace un escaneo de la carpeta `/agentes`.

- Si encuentra un archivo llamado `agente_codificador.yaml`, lo carga.
- Si le vas a vender el sistema a un contador, simplemente vas a esa carpeta, **borras el archivo `agente_codificador.yaml`**, y agregas uno nuevo llamado `agente_tributario.yaml`. Al encenderse, el sistema ni siquiera sabrá que alguna vez existió un agente programador.

B. Archivos de Configuración (Manifiestos) Cada módulo, agente o herramienta tiene un pequeño archivo de texto (un manifiesto) que dice qué hace.

YAML

```
# Ejemplo del archivo del Agente Financiero
nombre: "Asistente Financiero"
descripcion: "Maneja cuentas y lee recibos"
herramientas_permitidas: ["leer_correo", "escribir_excel"]
```

Si tu cliente no quiere que el sistema lea sus correos, abres este archivo, borras `"leer_correo"` de la lista, y la IA pierde instantáneamente esa capacidad. Es literalmente modularidad para no-programadores.

C. Instaladores Basados en "Perfiles" Cuando empaquetes tu sistema para venderlo, crearás "Perfiles de Instalación".

- **Perfil Tech/CTO:** Instala todos los módulos de programación, GitHub y terminal.
- **Perfil Pyme/Comercial:** Instala solo el Agente Financiero, el CRM, el Bus de Eventos y el Bot de Telegram.
- El instalador simplemente copia las carpetas correspondientes en el PC del cliente. Como todo funciona con LiteLLM y SQLite local, el cliente solo pone su propia tarjeta de crédito para la API y sus datos nunca salen de su máquina.

D. El Dashboard Dinámico Como la interfaz web (el Front) lee directamente lo que hay en las carpetas, si borras el agente de código, el "widget" de código desaparece automáticamente del Dashboard web. No tienes que reprogramar la interfaz visual para cada cliente.

Con estas adiciones (CRM, Presencia, Guardia, Eventos) y esta arquitectura de **Carpetas Dinámicas**, tienes un producto que no solo es un traje a la medida para gestionar tus propios desarrollos y proyectos de arte o educación, sino un software B2B (Business to Business) completamente franquiciable.

Módulo Core: OpenCode como Motor de Orquestación Base

1. Justificación de la Arquitectura

Para el desarrollo del **AI-OS de VirtualBrain SpA**, no se construirá un orquestador de agentes desde cero. En su lugar, el sistema utilizará el proyecto de código abierto **OpenCode** como su infraestructura base (el "Arnés" central).

Al aprovechar una arquitectura multi-agente ya probada y nativamente compatible con el estándar MCP (Model Context Protocol), el desarrollo se mantiene ágil. Esto permite construir y acoplar módulos complejos puramente mediante *vibe coding*, enfocando el esfuerzo en la lógica de negocio y la creación de herramientas, sin desgastarse en la resolución de conexiones de red o la gestión de dependencias de bajo nivel. OpenCode resuelve de fábrica la neutralidad de modelos, permitiendo alternar entre APIs de vanguardia (como Gemini Pro o Google Antigravity) y modelos de código abierto locales (como Qwen) según la necesidad de la tarea.

2. Plan de Implementación (A grandes rasgos)

Para transformar OpenCode en el "Cerebro" específico de la empresa y en un producto empaquetable, realizaremos cuatro adaptaciones estructurales:

A. Hackeo y Especialización de Agentes (Repurposing) OpenCode ya posee una jerarquía de agentes primarios y subagentes. En lugar de crear un enjambre nuevo, reconfiguraremos los existentes:

- El agente técnico por defecto (*Build*) se mantendrá con permisos elevados para actuar como el "CTO Adjunto", encargado de la automejora del sistema, la estructuración de bases de datos y la codificación de nuevos módulos interactivos o de software.
- Se clonarán subagentes (como el de investigación general) para crear perfiles altamente especializados: un **Agente Financiero** y un **Agente Comercial (CRM)**, aislándolos para que solo tengan acceso a sus respectivos *skills*.

B. Inyección de Skills (Herramientas Locales y Móviles) Se aprovechará la arquitectura de *plugins* y comandos de OpenCode para conectarle las "manos" del sistema.

Desarrollaremos pequeños scripts atómicos que se registrarán en el sistema como nuevas capacidades:

- Un *skill* de conexión local a WhatsApp para enviar y recibir notas de voz o instrucciones remotas.
- *Skills* de lectura de correo (Gmail API) y automatización de presupuestos integrados a bases de datos SQLite locales.
- Conexiones a bases vectoriales (ChromaDB) para alojar manuales técnicos y documentaciones de proyectos.

C. La Capa Visual: El Wrapper (Frontend) OpenCode es nativo de la terminal de comandos, un entorno hostil para un usuario final. Para solucionar esto, el motor de OpenCode se encapsulará y se ejecutará de forma invisible en el *backend*. Por encima, se construirá un **Dashboard Web Modular** (HTML/CSS/JS local) que se comunicará con OpenCode. Esta interfaz se diseñará con una estética oscura, melancólica y lofi, proporcionando un espacio de trabajo visualmente inmersivo, onírico y de baja fricción, donde los agentes se presentan como "widgets" o bloques de información.

D. Empaquetado y Marca Blanca (White-Labeling) Para cumplir con el objetivo de distribuir el AI-OS a colaboradores o clientes que no tengan conocimientos técnicos, se creará un "Perfil de Producción" a partir de nuestro *fork* de OpenCode. En esta versión empaquetada:

- Se "borrarán" (desactivarán) el agente *Build* y las herramientas de terminal para evitar que el usuario modifique accidentalmente el código del sistema.
- Se entregará únicamente el ejecutable con el Dashboard Visual, el Agente Financiero, el CRM y la conexión a WhatsApp. Cada instalación operará como un silo independiente, garantizando la privacidad absoluta de los datos locales del cliente.